

CS213 - Practice Quiz Questions

Question 1

The following postfix expression with single-digit operands is evaluated using a stack:

$8\ 2\ 3\ ^\wedge / 2\ 3\ ^* + 5\ 1\ ^* -$

Note: $^\wedge$ is the exponentiation operator.

What are the top two elements of the stack after the first * is evaluated?

A. 6, 1

B. 5, 7

C. 3, 2

D. 1, 5

Answer: A. 6, 1

Question 2:

Two stacks share a single array of size N : Stack A grows from index 0 upward, while Stack B grows from index $N - 1$ downward. Which of the following statements are correct?

A. Overflow occurs when $\text{topA} + 1 == \text{topB}$ and an item is pushed into either stack.

B. This scheme wastes less space than giving each stack a fixed half of the array when one stack is usually much smaller than the other.

C. If Stack B remains empty, Stack A alone can grow to hold all N elements.

D. An element can be pushed into Stack A whenever $\text{topA} < N - 1$, regardless of the current position of topB .

Answer: A, B, C

Question 3:

A stack is initially empty. The elements 1, 2, 3, 4, 5 are pushed onto the stack in this order. You may perform a pop operation at any time after an element has been pushed, and you may alternate between push and pop operations as desired. However, the elements must be pushed in the order $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

Which of the following cannot be obtained as the order of popped elements?

- A. 5, 4, 3, 2, 1
- B. 2, 1, 4, 3, 5
- C. 2, 4, 1, 3, 5
- D. 3, 2, 1, 5, 4

Answer: C

Question 4

Consider the following code:

C/C++

```
struct node { int val; node* next = nullptr; };

node* head = new node{3};
node* tail = head;

tail->next = new node{2};
tail = tail->next;

tail->next = new node{1};
tail = tail->next;
```

```
head = new node{0, head};
```

What is the final linked list?

A. $1 \rightarrow 2 \rightarrow 3 \rightarrow 0$

B. $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

C. $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

D. $0 \rightarrow 3 \rightarrow 2 \rightarrow 1$

Answer: D

Question 5

Which of the following statements are true for a singly linked list?

A. Nodes must be stored in contiguous memory locations.

B. Insertion at the beginning can be done in $O(1)$ time.

C. If a pointer to a node p is given, insertion immediately after p takes $O(1)$ time.

D. Searching for an element takes $O(1)$ time.

E. The last node typically stores nullptr in its next field.

Answer: B, C, E

Question 6

What will be the output of the following code:

C/C++

```
struct Node { int val; Node* next; };

Node* head = nullptr;

for (int i = 1; i <= 4; i++) {

    Node* n = new Node{i, head};

    head = n;

}

Node* cur = head;

int sum = 0;

while (cur) {

    if (cur->val % 2 == 0) sum += cur->val;

    cur = cur->next;

}

cout << sum;
```

- A. 4
- B. 6
- C. 10
- D. 3

Answer: B. 6

Question 7

Which of the following statements about a doubly linked list (each node

storing both `next` and `prev` pointers) are TRUE?

- A.** A node in a doubly linked list requires more memory per node than a node in a singly linked list, because of the extra pointer.
- B.** Given a pointer to a node (not necessarily the head), that node can be deleted in $O(1)$ time, without traversing the list from the head.
- C.** A doubly linked list can be traversed in both the forward and the backward direction.
- D.** Insertion at the head of a doubly linked list is $O(n)$, because both the new node's pointers and the old head's `prev` pointer must be updated.
- E.** Using a doubly linked list instead of a singly linked list eliminates the possibility of memory leaks.

Answer: A, B, C

Question 8

A circular queue is implemented with an array of size n , along with two indices `begin` and `end`, and using the condition $(end + 1) \% n == begin$ to detect a full queue (without any other variables). Which of the following statements are true about this design?

- A.** The queue can hold at most $n-1$ elements at any time
- B.** When `begin == end`, the queue could be either empty or full, so this design is buggy
- C.** When the array is completely filled, one slot is deliberately left empty to distinguish full from empty
- D.** This design wastes no space compared to a circular queue that uses a separate flag to indicate whether the queue is full

Answer: A, C

Question 9

What will be the output of the following code?

```
C/C++
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;
    for (int i = 1; i <= 5; i++) q.push(i);

    int n = q.size();
    for (int i = 0; i < n; i++) {
        int val = q.front();
        q.pop();
        if (val % 2 == 0) {
            q.push(val);
        }
    }

    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }

    return 0;
}
```

Answer: 2 4

Question 10

A queue is implemented using two stacks, **s1** and **s2**, so that enqueue is $O(1)$ and dequeue is amortized $O(1)$. Enqueue always pushes onto **s1**. For dequeue: if **s2** is empty, pop every element from **(a)** and push it onto **(b)**, then pop and return the top of **s2**. If **s2** is not empty, simply **(c)**.

Answer: (a) s1, (b) s2, (c) pop and return the top of s2

Question 11

What will be the output of the pseudocode?

None

```
Q = [10, 20, 30, 40] // 10 is at the front
```

```
while Q is not empty:
```

```
    x = dequeue(Q)
```

```
    if x % 20 == 0:
```

```
        print(x)
```

```
    else:
```

```
        enqueue(Q, x + 5)
```

- A) 20 40 10 30
- B) 10 20 30 40
- C) 20 40 20 40
- D) The loop runs infinitely

Answer: C) 20 40 20 40

Question 12

Which problem of a linear array queue is solved by a circular queue?

- A) Underflow
- B) Overflow
- C) Wasted space at the beginning
- D) Slow searching

Answer: C

Question 13

Which statements about queue overflow are correct?

- A) It can occur when inserting into a full bounded queue.
- B) It is the same as underflow.
- C) A circular queue can experience overflow.
- D) A dynamically allocated linked-list queue can never run out of memory.

Answer: A, C

Question 14

A deque of size 5 is implemented using a circular array. Initially, it is empty. Perform these operations:

```
None  
push_back(10)  
push_back(20)  
push_back(30)  
pop_front()  
push_front(40)  
push_back(50)  
pop_back()  
push_front(60)  
push_back(70)
```

What will be the final order of elements from front to rear? *[Single Correct Option]*

- A. 60, 40, 20, 30, 70
- B. 40, 20, 30, 60, 70
- C. 60, 40, 20, 70
- D. 40, 20, 30, 50, 60

Answer: A

Question 15

Which of the following statements about a Deque (Double-Ended Queue) are correct? *[Multiple Correct Option]*

- A. Insertion and deletion can be performed from both ends.
- B. A deque can be used to implement both a stack and a queue.
- C. A deque always follows FIFO order.

Answer: A,B

Question 16

Consider the following pseudocode:

```
None
D = empty deque

for i = 1 to 5:
    if i is odd:
        D.push_front(i)
    else:
        D.push_back(i)

while D is not empty:
    print(D.pop_and_get_front())
```

What will be the output? [Single Correct Option]

- A. 1 2 3 4 5
- B. 5 3 1 2 4
- C. 4 2 1 3 5
- D. 5 4 3 2 1

Answer: B

Question 17

You need to find the **maximum value in every window of size K** in an array of **N** elements. In the **$O(N)$** solution seen in class, which properties were relied up on?

- A. Each index is inserted into the deque at most once.

- B. Each index is removed from the deque at most once.
- C. Values corresponding to indices in the deque are maintained in decreasing order.
- D. The deque must always contain exactly K elements.

Answers: A, B, C

Question 18

Consider the following array:

$A = [1, 3, -1, -3, 5, 3, 6, 7]$

Let the **sliding window size be $K = 3$** .

You want to find the maximum element in every window using a **deque of indices**. The deque is maintained according to these rules:

1. The indices in the deque are always within the current window.
2. The corresponding array values are maintained in **decreasing order** from the front to the back.
3. Before inserting a new index i , remove indices from the **back** while their corresponding values are smaller than $A[i]$, since they can never become the maximum of any future window containing i .
4. If the index at the **front** falls outside the current window, remove it.
5. The index at the front therefore always represents the maximum element of the current window.

After processing **all elements of the array**, which of the following correctly represents the indices remaining in the deque?

Note: Use **0-based indexing**, i.e., $A[0] = 1, A[1] = 3, \dots, A[7] = 7$.

- A. $[4, 6, 7]$
- B. $[4, 5, 6, 7]$
- C. $[6, 7]$
- D. $[7]$

Answer: D. $[7]$

Question 19

A system maintains tasks in a deque.

- High-priority tasks are inserted at the **front**.
- Normal-priority tasks are inserted at the **back**.
- The scheduler always executes from the **front**.
- A running task may be moved to the **back** if it does not finish within its time quantum.

Which properties make a deque suitable for this scheduler?

- A. Insertion at both ends is $O(1)$.
- B. Removal from both ends is $O(1)$.
- C. It supports efficient round-robin behaviour.
- D. Random access to any task is $O(1)$.

Answer: A, B, C

Question 20

Here is an attempt at reversing a singly linked list in place.

```
struct node { int val; node* next = nullptr; };
```

```
node* reverse(node* head) {  
    node* prev = nullptr;  
    node* curr = head;  
    while (curr != nullptr) {  
        curr->next = prev;  
        prev = curr;  
        curr = curr->next;  
    }
```

```

}

return prev;

}

```

It is called as `head = reverse(head)` on the list $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, where `head` points to the node holding 1. What happens? [Single Correct Option]

- A. It returns $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$, so the reversal is correct.
- B. It returns a list holding only 1, and the nodes holding 2, 3 and 4 can no longer be reached.
- C. The loop never ends, because `curr` keeps moving between the first two nodes.
- D. The program follows a null pointer on the second iteration and crashes.

Answer: B

Question 21

Floyd's cycle detection algorithm from class is run on a singly linked list that may or may not contain a cycle.

```

node* slow = head; node* fast = head;

while (fast && fast->next) {

    slow = slow->next;

    fast = fast->next->next;

    if (slow == fast) return true; // cycle!

}

return false; // no cycle

```

Which of these statements are true? [Multiple Correct Options]

- A. If the list has a cycle, the two pointers are certain to meet, because once both are inside the cycle `fast` gains one node on `slow` every iteration.
- B. If the list has no cycle, the loop ends after $\Theta(n)$ iterations and the whole check uses $O(1)$ extra space.
- C. If the pointers do meet, they meet at the first node of the cycle.

D. Once the pointers meet, moving one of them back to head and then advancing both one node at a time makes them meet at the first node of the cycle.

E. Detecting a cycle for certain needs $\Theta(n)$ extra space, such as a hash set of the node addresses seen so far.

Answer: A, B, D

Question 22

The code below is run on a singly linked list with at least one node. Number the nodes 1, 2, ..., n starting from the head.

```
node* slow = head;

node* fast = head;

while (fast->next && fast->next->next) {

    slow = slow->next;

    fast = fast->next->next;

}

cout << slow->val;
```

(i) What does it print for the list $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60$?

(ii) For a general n, which node does slow stop at? [Single Correct Option]

A. 30; in general node $\text{ceil}(n/2)$. For even n that is the first of the two middle nodes.

B. 40; in general node $\text{floor}(n/2) + 1$. For even n that is the second of the two middle nodes.

C. 30; in general node $\text{floor}(n/2)$.

D. Nothing is printed. For even n the loop follows a null pointer when it evaluates fast->next->next , so the program crashes.

Answer: A